

Level 5. Design questions, worked end to end

This is the top level. Levels 1 to 4 gave you the words (component, interface, data flow, trade-off), the common robot boxes, the three questions to ask about any arrow (latency, throughput, failure), and how to make a design safe. This level puts all of it to work on six real robots, the way you would if someone handed you a blank whiteboard and forty minutes.

It is also the only level that talks about hiring. A **design question** is a conversation in which someone gives you a vague request ("design a robot that does X") and watches how you turn it into boxes, arrows, numbers and trade-offs. Nobody expects a finished design. They want to see a method. This level teaches the method, then applies it six times so you can see what "good" looks like at the end of each step. Read it as coaching, not as lines to recite.

Rough time: two evenings for the method and the first three designs, one more for the rest.

5.1 The method

Forty minutes is short. The method below is seven steps with a time for each. The times add up to forty. Most people who struggle with design questions fail step 1 or step 2 and then run out of time drawing boxes for the wrong robot.

Step 1	Step 2	Step 3	Step 4	Step 5	Step 6	Step 7
Ask what it must do and never do	Numbers first	Draw sense -> think -> act	Label the arrows	Go deep on the 3 hardest arrows	Name the trade-offs	Say what to build first
~5 min	~5 min	~5 min	~5 min	~12 min	~5 min	~3 min

Step 1. Ask what the robot must do and must never do (5 minutes). The request is vague on purpose. Before you draw anything, narrow it with five questions. Write the answers in a corner of the board and point back at them all the way through.

1. **Who uses it and where?** A warehouse with trained staff is not a pavement with children.
2. **What does "done" look like?** Picks per hour, deliveries per day, blades per battery. One number you can be measured against.
3. **What must it never do?** Hit a person, drop a parcel in the road, fly into the blade, move when the surgeon has not asked it to. This is the safety list. Every design later gets checked against it.
4. **What is fixed and what is free?** Is the hardware chosen already? Is there a budget? Is there a deadline? Is there a network?
5. **What happens today without the robot?** The manual process tells you the real requirements. A human picker in a bin takes a few seconds per item; that is your target.

If the person asking says "you decide", decide, say the assumption out loud once, and move on. Do not spend ten minutes on questions.

Step 2. Numbers first (5 minutes). Before boxes, write down five numbers. Speed of the robot. Size of the space it works in. The rate of the main sensor. The battery and how long a shift is. The budget or the number of units. Each number will decide something later: speed decides stopping distance, which decides sensor range, which decides sensor cost. Rough is fine. "About 1 metre per second" is a number. "Fast" is not.

Step 3. Draw sense, think, act (5 minutes). Left to right, as in Level 1. Start with three boxes and split each one only when you have a reason. Six to ten boxes is the right size. Group anything bigger into a box with a name and say you will open it if asked.

Step 4. Label the arrows (5 minutes). For every arrow: what travels and how often. An image at 10 Hz. A pose at 50 Hz. A goal on change. This is where you catch the arrow that carries 500 megabytes a second over Wi-Fi, before you build it.

Step 5. Pick the three hardest arrows and go deep (12 minutes). Not all arrows are equal. Find the three that decide whether the robot works: usually one about latency (the loop that stops the robot), one about failure (what stops working when the network drops), and one about scale (what breaks when there are two hundred robots instead of one). For each, say the number, say what happens when it is missed, and say the fix.

Step 6. Name the trade-offs (5 minutes). For each big choice say "X because Y matters more than Z here". Central planner because global optimality matters more than the single point of failure, which we cover with a hot standby. Cheaper lidar because the robot walks at 1.5 m/s and never needs to see past 10 metres. Three or four of these is plenty.

Step 7. Say what you would build first (3 minutes). A design nobody can start on is not a design. Name the thinnest version that proves the riskiest part: one robot, one route, a remote stop button. Then say what you would add second and third.

The one-page checklist

- ☐ 5 questions answered, written in the corner (who, done, never, fixed, today)
- ☐ 5 numbers written down (speed, space, sensor rate, battery, budget)
- ☐ sense -> think -> act, 6 to 10 boxes, left to right
- ☐ every arrow labelled: what + how often
- ☐ 3 hardest arrows chosen: one latency, one failure, one scale
- ☐ for each hard arrow: the number, what happens if missed, the fix
- ☐ 3 or 4 trade-offs said as "X because Y matters more than Z"
- ☐ safety is on the robot, not in the cloud
- ☐ what to build first, second, third
- ☐ checked the design against the "must never" list

Each of the six designs that follow uses exactly these steps as headings, so you can see the method applied rather than described.

5.2 A robot arm picking items from a bin with one camera

This is exercise 1 from Level 1. The request: "Design a robot arm that picks items out of a bin and puts them in a box. There is one camera above the bin."

Step 1. What it must do and never do

Who and where: a warehouse packing station. Trained staff walk past, sometimes reach into the bin to fix a jam. Done means about 600 picks an hour with less than one jam an hour that needs a person. Never: touch a person, crush an item, drop an item outside the box. Fixed: a six-joint arm with a suction gripper and one colour-plus-depth camera (a camera that gives a colour image and, for every pixel, a distance) are already bought. Today: a human picks at roughly one item every four seconds, so 600 an hour (one every six seconds) is acceptable.

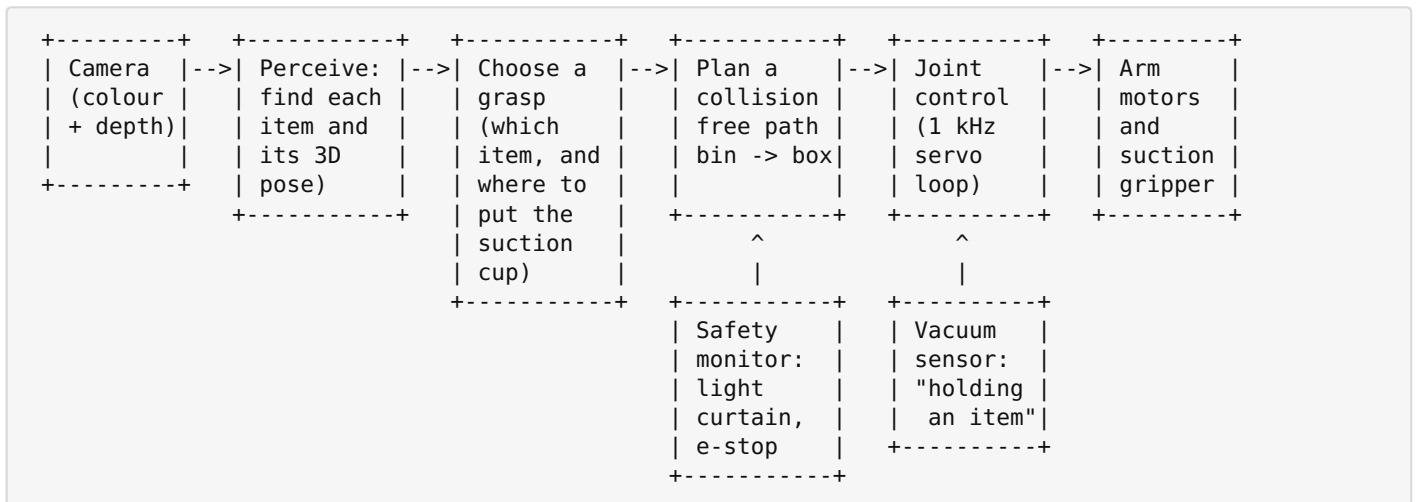
Step 2. Numbers first

Number	Value	Why it matters
Arm tool speed	up to 1 m/s	sets stopping distance and the safety zone

Number	Value	Why it matters
Bin to box distance	0.8 m	sets the travel time per pick
Camera	1280 by 720 colour plus depth, taken once per pick	4.6 MB per capture
Pick target	600 per hour, one every 6 s	the budget every step must fit inside
Joint control loop	1000 times a second (1 kHz)	the arm's inner loop, 1 ms per step

One capture is 1280 by 720 pixels times 3 bytes for colour (2.8 MB) plus 2 bytes per pixel for depth (1.8 MB). If you streamed it at 10 Hz that would be 46 MB/s. You do not need that. One capture per pick, six a minute, is almost nothing.

Step 3. Draw sense, think, act



The **light curtain** is a row of light beams at the edge of the cell; a hand breaking a beam is a signal. The **safety monitor** is a separate small box whose only job is to stop the arm. It is not part of the perception computer, on purpose (Level 4).

Step 4. Label the arrows

Arrow	What travels	How often
Camera → Perceive	colour and depth image, 4.6 MB	once per pick, about every 6 s
Perceive → Choose grasp	list of items: 3D position, orientation, a "how sure" score	once per pick
Choose grasp → Plan path	one grasp pose (x, y, z, angle) and one drop pose	once per pick
Plan path → Joint control	a path: list of joint angles with times, about 200 points	once per pick
Joint control → Motors	a torque or velocity per joint	1 kHz
Motors → Joint control	actual joint angle and current per joint	1 kHz
Vacuum sensor → Choose grasp	"item held" or "lost it"	100 Hz
Light curtain → Safety monitor	"beam broken"	immediately, wired, under 10 ms

Arrow	What travels	How often
Safety monitor → Joint control	"stop now"	immediately, a physical line, not a message

Step 5. The three hardest arrows

Latency: the light curtain to a stopped arm. The tool moves at up to 1 m/s. From a hand breaking the beam to the arm being still: detection 50 ms, controller reaction 100 ms, braking 200 ms, total 0.35 s. In that time the arm can travel 0.35 m. A person's hand moving in at 1.6 m/s (the standard figure used for safety distance) covers 0.56 m. So the light curtain must sit at least $0.35 + 0.56$, about 0.9 m, outside the furthest point the arm can reach. Fix if there is not that much room: slow the arm when a person is near (a zone sensor), which shrinks the 0.35 m.

Failure: the perception box returns rubbish. A shiny bag gives the depth camera no reading. A transparent item is invisible. The grasp chooser must have a "how sure" score on every item and refuse to pick under a threshold. After two refusals on the whole bin, it shakes the bin (a small tilt) and retries once, then calls a person. With 98% grasp success at 600 picks an hour you still get 12 failures an hour, so the recovery path is not an edge case, it runs every five minutes.

Scale: the pick cycle budget. Capture 0.1 s, perceive 0.3 s, choose 0.1 s, plan 0.2 s, move to grasp 1.5 s, grasp 0.3 s, move to box 1.5 s, release 0.2 s, return 1.2 s. That is 5.4 s, which gives 666 picks an hour, just above the target. The two 1.5 s moves dominate. The trick that buys time: take the next picture while the arm is over the box, so capture and perception of pick N+1 overlap with the move of pick N. That takes 0.4 s out of the cycle.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Suction cup, not fingers	most parcels are flat boxes and bags, and suction is fast and simple	cannot pick mesh bags or heavy items; needs the vacuum sensor
One capture per pick, not a video stream	freshness is not needed between picks, and it keeps the compute small	if a person nudges the bin mid-cycle the picture is stale, so the vacuum sensor catches the miss
Separate safety monitor with wired stop	a person's safety must not depend on the perception computer being healthy	a second box, more wiring, more cost
Slow zone instead of a full cage	staff need to reach in often; a cage would be opened all the time	needs a zone sensor and testing to prove the slow speed is safe

Step 7. What I would build first

One arm, one bin, one item type, a fixed drop point, a full physical fence with a door interlock (the arm cannot move while the door is open). No grasp chooser at all: pick the item whose centre is highest. Measure the real success rate and cycle time. Then add: the "how sure" score and the recovery path, then mixed item types, then the light curtain and slow zone to replace the fence.

Questions someone might push on, with answers

- *Why not put the camera on the arm?* A wrist camera sees around occlusions but you have to move the arm to take a picture, which costs cycle time. One fixed camera above the bin is faster. Add a wrist camera later if the fixed one keeps missing items at the bin's edges.
- *Does this need ROS 2?* No. The boxes and arrows exist without it. ROS 2 gives you a **node** (a running box), a **topic** (a named stream, like the image arrow), a **service** (ask and wait, like "plan a path"), and

an **action** (a long job with progress, like "move to the box"). It would be a natural fit for the perception to control side. The 1 kHz joint loop and the safety monitor would not go through ROS 2 messaging.

- *Where is the collision map for the planner?* The bin walls and the box are fixed and known. Items in the bin come from the depth image. A planner such as the A* in code/03-grids-bfs-dfs/astar works on a grid; an arm planner works in joint space, but the idea of "search from start to goal, avoiding blocked cells" is the same.

5.3 A pavement delivery robot

Exercise 2 from Level 1. The request: "Design a robot that drives on pavements from a shop to a house with a bag of groceries." This one ties Levels 2, 3 and 4 together: the standard boxes of a mobile robot (Level 2), the latency, throughput and failure questions on each arrow (Level 3), and the safety layer that sits outside the clever software (Level 4).

Step 1. What it must do and never do

Who and where: a town centre pavement shared with pedestrians, prams, dogs and bikes. Done: deliver within 30 minutes over routes up to 2 km, 20 deliveries a day per robot, and the customer opens the lid with a code. Never: hit a person, roll into the road, tip over on a kerb, get stuck where nobody can reach it, lose the goods. Fixed: a six-wheeled base with a lockable box, 4G mobile data, a budget that rules out a car-grade lidar. Today: a cyclist does this, and takes about 10 minutes for 2 km.

Step 2. Numbers first

Number	Value	Why it matters
Speed	1.5 m/s (5.4 km/h, brisk walking)	stopping distance, and how far a delayed command lets it travel
Route	2 km, so 22 minutes at 1.5 m/s	fits the 30 minute promise with time for crossings
Battery	500 watt-hours, average draw 150 W	3.3 hours, which is 18 km of driving, so about 4 round trips
Cameras	2 front cameras, 1280 by 720, 15 Hz	83 MB/s raw, must be processed on the robot, never sent raw
Lidar	small 16-beam, 300,000 points a second	4.8 MB/s, sees 2 to 20 m, enough at walking speed
4G link	100 to 300 ms round trip, a few Mbit/s up	rules out remote driving, allows remote supervising

Step 3. Draw sense, think, act

```

+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| Cameras |->| Perceive: |->| Local  |->| Path  |->| Motion |->| Wheel  |
| Lidar   | | people,  | | map:   | | planner: | | control | | motors  |
+-----+ | kerbs,   | | free   | | route + | | (speed, | +-----+
+-----+ | crossings | | space  | | avoid  | | turn)   |
| Wheel   | +-----+ | around me | +-----+
| encoders|->+-----+ +-----+ ^ ^
| IMU     | | Localise: |-----+ |
| GPS     | | where am | +-----+ +-----+ +-----+
+-----+ | I on the  |->| Mission: | | Remote | | Safety |
          | route map | | which   | <-| supervisor| | monitor: |
          +-----+ | delivery, | | (human, | | bumper, |
                   | next stop | | via 4G) | | tilt,   |
                   +-----+ +-----+ | lidar  |
                                           | near   |

```

```
| field |
+-----+
```

IMU is an inertial measurement unit: a chip that measures turn rate and acceleration. **GPS** gives a position from satellites, good to a few metres, worse between buildings. **Localise** combines encoders, IMU, GPS and what the cameras see against a stored route map, to answer "where am I" to about 0.2 m; this is the Kalman filter idea in code/06-robotics/01_kalman_1d.cpp with more inputs. The **route map** is made once by driving the route with a person: a grid of the pavement 2 km long and 20 m wide at 0.1 m cells is 4 million cells, 4 MB at one byte each, small enough to live on the robot.

Step 4. Label the arrows

Arrow	What travels	How often
Cameras → Perceive	two images, 1280 by 720	15 Hz
Lidar → Perceive and Safety monitor	point cloud, 300k points/s	10 Hz sweeps
Encoders, IMU, GPS → Localise	wheel ticks 100 Hz, turn rate 200 Hz, position 10 Hz	as listed
Perceive → Local map	boxes around people and bikes with speed; kerb line	15 Hz
Localise → Path planner	(x, y, heading) on the route map, with a "how sure"	20 Hz
Mission → Path planner	the next waypoint on the route	on change
Path planner → Motion control	target speed and turn rate for the next 2 s	20 Hz
Motion control → Motors	a speed per wheel	50 Hz
Robot → Remote supervisor	compressed video 2 Mbit/s, position, state	video continuous, state 1 Hz
Remote supervisor → Mission	"go", "wait", "take the other path", "stop"	on change, seconds apart
Safety monitor → Motion control	"stop", overriding everything	wired, immediate

Step 5. The three hardest arrows

Latency: seeing a child to being stopped. Perception takes 100 ms, planning 50 ms. In that 0.15 s the robot travels 0.22 m. Braking from 1.5 m/s at 3 m/s squared takes 0.5 s and 0.38 m. Total stopping distance 0.6 m. So anything the lidar sees inside 1 m must already be triggering a stop, and the near-field check lives in the safety monitor, not in the perception pipeline that takes 100 ms. The Level 3 rule: the fastest, dumbest check gets the shortest path to the motors.

Failure: the 4G link drops. It will, several times a trip, under bridges and in dips. The robot must be fully able to drive without it: localisation, planning and the safety monitor all run on board. The link is only for the supervisor's decisions at hard spots (a road crossing, a blocked pavement). Rule: if the link has been silent for 10 s and the robot is at a decision point, it waits at the kerb. If it is mid-pavement it carries on to the next safe waiting spot. A supervisor watching 2 Mbit/s video over a 22 minute trip uses 330 MB of data, so the video is sent only at decision points and on request, not all the time.

Scale: one supervisor for many robots. A supervisor cannot drive robots: 300 ms round trip is 0.45 m of travel between a picture and a command. So the robot drives, and the supervisor only answers "may I cross?" and "which way round this van?". Each such question takes a human about 20 s. If one robot asks twice per trip, that is 40 s of attention per 22 minute trip, so in theory one supervisor can handle 33 robots. Questions cluster at busy times, so plan for 20 to 30. That number is the business case, and it follows straight from the latency arrow.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Walking speed, not cycling speed	stopping distance stays under 1 m, so a cheap short-range lidar is enough	22 minutes for 2 km, so fewer trips per day
All driving on board, supervisor only at decision points	the link is unreliable and slow, and safety must not depend on it	a more capable, more expensive computer on every robot
A pre-driven route map, not a general city map	localisation against a known route is far easier and more reliable	every new route needs a mapping drive first
Wait at the kerb when unsure	an idle robot is a small cost; a wrong crossing is a huge one	robots pile up at crossings on busy days; the supervisor needs a queue

Step 7. What I would build first

One robot, one route, a person walking behind it with a remote stop. No perception of people at all: the safety monitor stops the robot for anything inside 1 m and it waits. Measure how often it stops and for how long. Then add: the people detector so it can steer around rather than stop, then the supervisor console and the crossing question, then the second route.

Questions someone might push on, with answers

- *What about road crossings without a signal?* The robot never decides to cross on its own at first. It asks the supervisor, who looks at the video. Later, when the perception has months of data, crossing at a zebra crossing with no cars in view can be automated, and the supervisor is told rather than asked.
- *How does localisation survive GPS going to 20 m between tall buildings?* The Kalman filter weights each input by how sure it is. GPS gets a large uncertainty in the city and the cameras matching the route map take over. That is why the route is pre-driven.
- *Where is the BFS or A here?** The path planner. The route map is a grid, the robot's cell is the start, the next waypoint is the goal, and cells with people or kerbs are blocked. That is exactly tutorial/03-grids-bfs-dfs.md with a cost for "near a person".

5.4 A drone inspecting a wind turbine blade for cracks

Exercise 3 from Level 1. The request: "Design a drone that flies up to a wind turbine and photographs each blade closely enough to find cracks."

Step 1. What it must do and never do

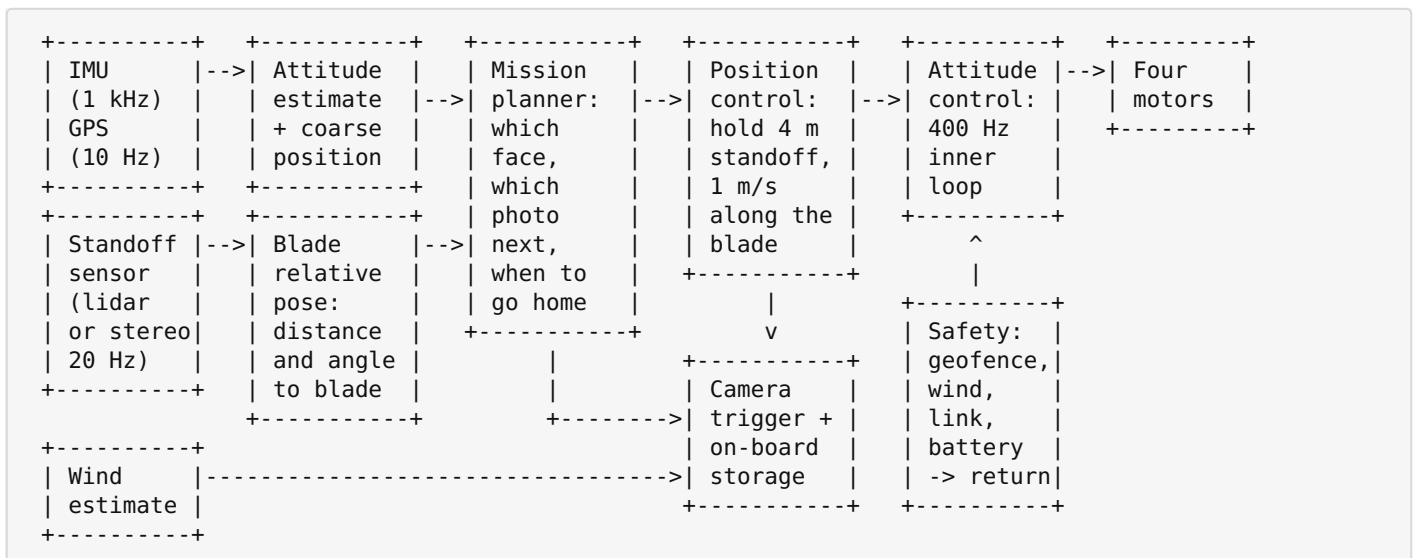
Who and where: an inspection team of two on a windy hill or on a boat, one turbine at a time, the turbine stopped and the blades parked. Done: every square metre of all three blades photographed at about 1 mm per pixel, cracks 3 mm and wider found, a report per blade. Never: touch the blade or tower, fly away (lose the link and keep going), fly in wind above its limit, lose the photos. Fixed: a commercial inspection drone with a 24 megapixel camera, 25 to 30 minutes of flight per battery. Today: a rope-access technician abseils the blade, which takes a day per turbine and is dangerous. A drone doing a turbine in under an hour is the target.

Step 2. Numbers first

Number	Value	Why it matters
Blade length	60 m; tower 100 m, so the tip is at 160 m	sets flight height and path length
Path	3 blades, 4 faces each (two sides, two edges), 720 m of flying	total time and photo count
Standoff	4 m from the blade	with a 24 mm lens on a full-frame sensor: 6 m by 4 m in frame, 1 mm per pixel
Photo spacing	70% overlap, so a new photo every 1.2 m; at 1 m/s that is 0.83 photos a second	600 photos per turbine
Storage	12 MB per photo, 7.2 GB per turbine	must be stored on board; 48 Mbit/s to stream it live, not available on a hill
Flight time	12 minutes of scanning plus 8 of transitions, 20 minutes	one battery per turbine, with no margin; plan for a swap
Wind	drone limit 12 m/s; gusts move it 0.5 m in a second	standoff must be held by a sensor, not by GPS

The number that decides the design is the last one. A gust that moves the drone 0.5 m towards a blade 4 m away is a near miss. The drone must measure its distance to the blade directly.

Step 3. Draw sense, think, act



Two loops, one inside the other. **Flight control** (attitude at 400 Hz, position at 50 Hz) keeps the drone level and where it is told to be. It exists in every drone and is usually bought, not written. **Mission planning** (1 Hz) decides where "where it is told to be" is: which face, how far along, when to trigger the camera, when to go home. Keep these in separate boxes and if possible on separate computers, because the mission code changes every week and the flight code must not.

Step 4. Label the arrows

Arrow	What travels	How often
IMU → Attitude estimate	turn rates and accelerations	1 kHz
GPS → Attitude estimate	position, good to 2 to 5 m, worse next to a steel tower	10 Hz

Arrow	What travels	How often
Standoff sensor → Blade relative pose	distance to the blade surface and its angle	20 Hz
Blade relative pose → Position control	"you are 4.3 m out, 5 degrees off square"	20 Hz
Mission planner → Position control	a target point along the face and a speed	1 Hz, held between updates
Mission planner → Camera trigger	"take a photo now" with the current pose	0.83 Hz
Camera → Storage	12 MB photo plus pose and time stamp	0.83 Hz, 10 MB/s to the card
Wind estimate → Mission planner and Safety	wind speed from how hard the motors work	1 Hz
Ground station → Mission planner	start, pause, abort, next blade	on change, a radio link
Safety → Position control	"return home now", overriding the mission	immediate

Step 5. The three hardest arrows

Latency and accuracy: the standoff sensor to position control. GPS cannot hold a 4 m gap next to a 100 m steel tower; it reflects the signal and the error grows to tens of metres. So the gap is held by a sensor pointed at the blade: a small lidar or a stereo pair, reading at 20 Hz. A gust pushes the drone 0.5 m in a second; at 20 Hz the control loop sees 0.025 m of that per reading and corrects. If the sensor loses the blade (looking past the tip, or at the sky), the position control must not go back to GPS and drift into the blade; it holds attitude, backs away 2 m, and asks the mission planner for a re-acquire.

Failure: the radio link or the battery. Every drone has a return-home rule; the question is what it returns *through*. From the far side of a blade at 160 m, straight home may go through the tower. The mission planner keeps a list of safe retreat points (out from the blade, then down) and the safety box uses them. Battery: 20 minutes of work against 25 to 30 of flight leaves no margin in wind, which raises the motor load. The rule is to return when the estimated time home plus 3 minutes exceeds the battery left, computed every second, not at a fixed percentage.

Scale: the photos. 600 photos, 7.2 GB, per turbine. Streaming them live would need 48 Mbit/s, which a hillside radio does not have. So the photos stay on the drone's card with the pose stamped on each, and are copied off between batteries. That means a lost drone is a lost turbine's work, and it means the report is built after landing, not in the air. The pose stamp is what lets the report say "crack at 23 m from the root on the pressure side".

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Hold standoff with a blade-facing sensor, not GPS	not hitting the blade matters more than anything	another sensor, and a loop to write and test
4 m standoff with 1 mm per pixel	finds 3 mm cracks, and 4 m absorbs a 0.5 m gust	600 photos and 20 minutes; 2 m would be sharper but a gust could reach the blade
Photos on the card, report on the ground	the radio cannot carry 48 Mbit/s and the drone's compute is better spent on flying	no live crack detection; the team waits for landing
Bought flight controller, own mission planner	the flight loop is a solved problem and certified; the mission is where the value is	limited to what the flight controller's interface allows
Stop the turbine and park the blades	a moving blade tip at 80 m/s cannot be flown near	a turbine out of service for an hour; the operator must agree

Step 7. What I would build first

The drone with the bought flight controller, flown by hand along one face while the mission planner only triggers the camera and stamps the pose. Check the photos are sharp (1/1000 s exposure at 1 m/s is 1 mm of blur, one pixel, acceptable) and the overlap is right. Then add the standoff sensor and the hold loop, tested on a wall. Then the automatic face-by-face path. Wind and battery rules go in before the first turbine, not after.

Questions someone might push on, with answers

- *Why not one wide photo of the whole blade from far away?* At 60 m away the same camera gives 15 mm per pixel; a 3 mm crack is invisible. Resolution forces you close.
- *Why 70% overlap?* So every point is in at least three photos, which lets the ground software stitch the images and see a crack from three angles; it also covers a missed trigger.
- *What is the geofence?* A volume the drone must stay in: a cylinder around the turbine, plus a floor and ceiling. Leaving it triggers return-home regardless of the mission. This is the Level 4 idea of a simple check outside the clever code.

5.5 A warehouse with 200 grid robots

The request: "Design the system that runs 200 robots on top of a grid of storage bins, the way Ocado's grocery warehouses work." The robots are small cubes that drive on rails over a grid; below each cell is a stack of crates; a robot lowers a hook, lifts the top crate, and carries it to a pick station where a person or an arm takes the items out.

Step 1. What it must do and never do

Who and where: an indoor hall, no people on the grid, engineers can enter only when the fleet is stopped. Done: 12,000 crates delivered to pick stations an hour, orders complete on time. Never: two robots in the same cell, a robot driving off a stopped section, a stuck robot blocking a lane for long, a crate dropped. Fixed: the grid is 100 by 60 cells, cells are 1 m, robots move at 2 m/s, one wireless radio system. Today: a bigger warehouse with people walking to shelves; the grid is the replacement.

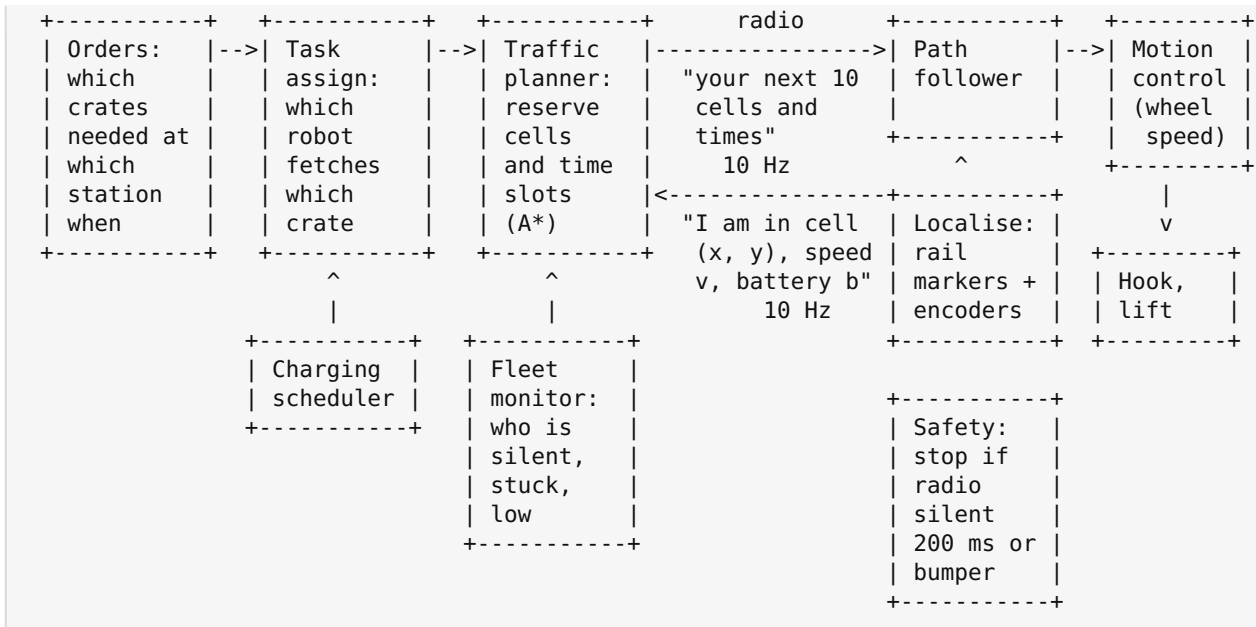
Step 2. Numbers first

Number	Value	Why it matters
Grid	100 by 60, 6000 cells, 1 m each	a path search over 6000 cells is tiny
Robot speed	2 m/s, so half a second per cell	the time slot the planner reserves per cell
Stopping	from 2 m/s at 4 m/s squared: 0.5 s, 0.5 m	a robot can stop inside one cell
Radio	each robot sends 50 bytes of status 10 times a second	200 robots is 100 kB/s, easy; the hard part is reliability, not volume
Battery	1 kWh, 300 W average draw, 3.3 hours running, 1 hour charging	23% of robots are charging at any time, so 260 robots for 200 working
Throughput	one crate per robot per minute	200 robots gives 12,000 an hour, the target

Step 3. Draw sense, think, act

CENTRAL (one computer, plus a hot standby)

EACH ROBOT (x200)



Task assignment decides which robot fetches which crate; a simple rule is "nearest idle robot", a better one looks a minute ahead. **Traffic planning** turns each task into a path and reserves every cell along it for a time window, so two robots are never promised the same cell at the same time. This is the A* in code/03-grids-bfs-dfs/astar with time added as a third dimension: a node is (x, y, t) instead of (x, y).

Step 4. Label the arrows

Arrow	What travels	How often
Orders → Task assign	a list of (crate id, station, deadline)	on change, tens a second
Task assign → Traffic planner	(robot id, from cell, to cell)	per task, about 3 a second across the fleet
Traffic planner → Robot	the next 10 cells with an arrival time for each	10 Hz, or when the plan changes
Robot → Central	cell, speed, battery, "carrying crate"	10 Hz, 50 bytes
Localise → Path follower	which cell and how far into it, from markers on the rails and wheel encoders	100 Hz
Path follower → Motion control	target speed for the next cell boundary	50 Hz
Fleet monitor → Traffic planner	"robot 37 silent" or "robot 37 stuck in (12, 40)"	on change
Charging scheduler → Task assign	"robot 37 goes to charger next"	on change
Safety → Motion control	"stop" if no valid plan received in 200 ms	immediate, on the robot

Step 5. The three hardest arrows

Failure: the radio goes silent for one robot. At 10 Hz, 200 ms of silence is two missed messages. The robot's own safety rule is to stop, and it can stop in 0.5 m, inside its current cell, which it holds a reservation for. In 200 ms it has moved only 0.4 m. So a silent robot is always inside a cell it was promised. That single rule is what lets the central planner be trusted: a robot never drives on a stale plan. The cost is that a noisy radio corner makes robots stop and start, and the planner routes around it after a few events.

Scale: the central planner's cycle. 200 robots, each replanned about once a second, each A* over a few thousand (x, y, t) nodes taking about a millisecond: 200 ms of planning per second on one core. That fits, with room. If it did not, the fix is to plan only the robots whose plan changed, and to give each robot 10 cells rather than the whole route so a change far away does not touch it.

Latency and correctness: the reservation table. Every cell has a list of (robot, from time, to time). A robot moving at 2 m/s spends 0.5 s in a cell; the planner reserves 0.5 s plus a 0.2 s margin either side, to cover clock error and stopping. If a robot is late (a slow wheel, a heavy crate) its later reservations are wrong. So the planner re-checks reservations against reported positions every cycle and shifts the ones behind schedule, which may push other robots. Clocks matter: all robots sync to the central clock so "arrive at t = 1042.3 s" means the same thing to everyone.

Step 6. Trade-offs

The big one is central versus distributed.

Choice	Chosen because	At the cost of
Central planner with reservation table	global view gives no deadlocks and near-optimal traffic; 200 ms per cycle is affordable	a single point of failure, so a hot standby that mirrors the table, and a fleet that stops if both fail
Distributed (each robot plans, negotiates with neighbours)	no single point of failure; scales past 1000 robots	deadlocks in crowded corners are hard to prove absent; that is why it is not chosen here
Robots stop on 200 ms silence	never driving on a stale plan matters more than the throughput lost to radio blips	needs a very reliable radio; the hall gets many access points
Nearest idle robot for assignment first	simple, explainable, and half the possible gain comes from it	a smarter assignment later can raise throughput 10 to 20%
Charge at 20% battery, opportunistically at 40% when idle	a robot that dies on the grid blocks a lane and needs a person	260 robots bought to have 200 working

Step 7. What I would build first

A simulator: the grid, 20 robots, the reservation table, and the A* in (x, y, t). Prove no two robots ever share a cell, and measure crates per hour. Then five real robots on a real corner of the grid with the same central code, then the radio-silence stop rule tested by unplugging an access point. Only then 200.

Questions someone might push on, with answers

- *What if a robot breaks down on the grid?* The fleet monitor sees no progress for 5 s and marks the cell blocked; the planner routes around; a recovery robot tows it, or the section is stopped for a person. A stuck robot in a busy lane costs minutes, so the layout has more lanes than the minimum.
- *Why not put the reservation table on the robots?* Because 200 copies must agree, and agreement over a lossy radio is a research problem. One table, one clock, one truth, with a standby.
- *How is this the A in the tutorial?** Same code shape: a priority queue of nodes, a cost so far, a heuristic (Manhattan distance on a grid), and neighbours. The only changes are that a node carries a time, "wait here" is a neighbour, and a neighbour is skipped if the reservation table says the cell is taken at that time.

5.6 A surgical robot arm teleoperated by a surgeon

The request: "Design a robot arm that a surgeon controls from a console, the way CMR Surgical's Versius or Intuitive's da Vinci work." **Teleoperation** means a person moves a control and the robot copies the motion; the robot decides nothing about where to go.

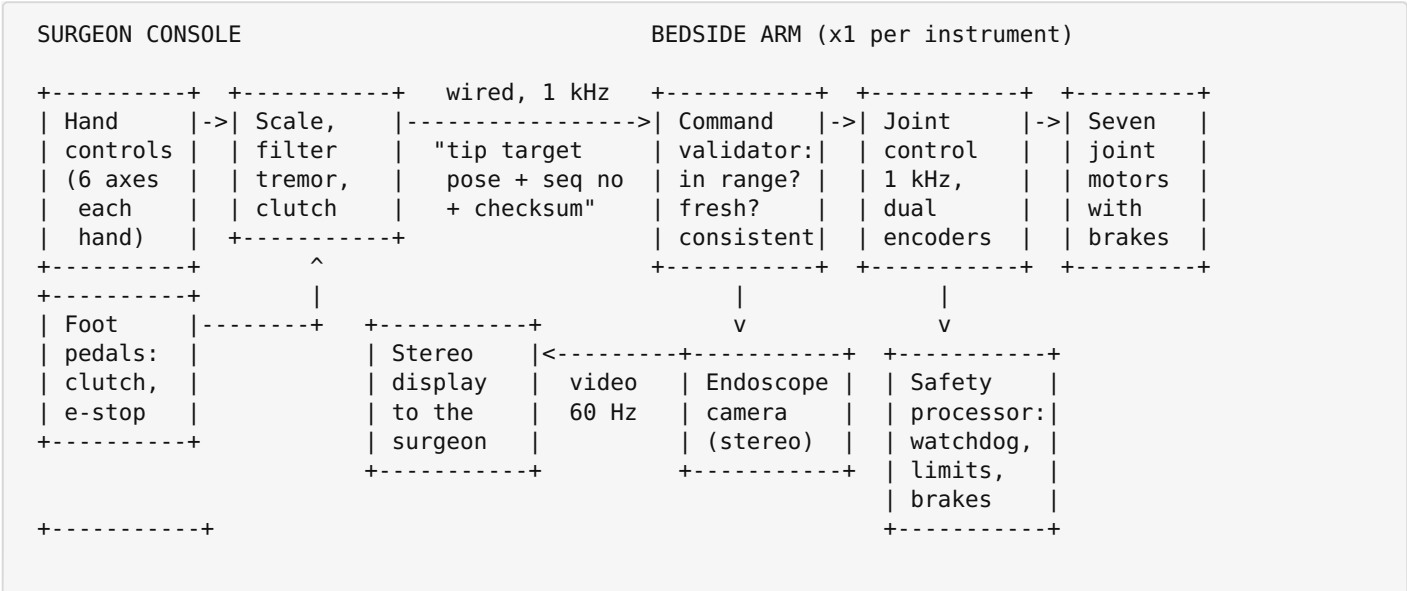
Step 1. What it must do and never do

Who and where: an operating theatre. The surgeon sits at a console a few metres from the patient; a nurse stands by the arm. Done: the instrument tip follows the surgeon's hand, scaled down and steadied, with a stereo (two-eye) video view that feels live. Never: move when the surgeon has not moved, move further or faster than commanded, keep moving after a fault, lose the record of what happened. Fixed: a seven-joint arm holding a thin instrument through a small port in the body; a wired network in the room, no internet in the loop. Today: the surgeon holds long instruments by hand; the robot adds scaling, tremor removal and a better view. It adds no autonomy at all, and that is a requirement, not a limitation.

Step 2. Numbers first

Number	Value	Why it matters
Command rate	1 kHz from console to arm	1 ms per step, so a stop is felt within a few ms
Motion scaling	3 to 1: a 3 cm hand move is a 1 cm tip move	hand at 10 cm/s gives a tip at 3.3 cm/s, 33 micrometres per 1 ms cycle
Tremor	human hand tremor is 8 to 12 Hz; filter cuts above 6 Hz	a 0.2 mm tremor becomes 0.07 mm at the tip and is then filtered
Video path	camera frame 16.7 ms (60 Hz) + encode 10 ms + display 16 ms, about 43 ms	the surgeon feels lag above roughly 100 ms; 43 ms leaves margin
Command path	sample 1 ms + wired network 1 ms + control 1 ms, 3 ms	the video is the slow side, not the commands
Watchdog	freeze if no valid command for 20 ms	the tip moves at most 0.67 mm before freezing at full speed
Logging	7 joints, 4 bytes each, 1 kHz: 28 kB/s, 400 MB per 4-hour operation; video 2 streams at 8 Mbit/s is 29 GB	both must be kept, for years

Step 3. Draw sense, think, act



```
| Audit log |<---- every command, every joint reading, every fault, both video streams
+-----+
```

The **clutch** is a pedal that disconnects hand from tip so the surgeon can reposition the hands, like lifting a mouse. The **command validator** checks each message: a sequence number so nothing is repeated or skipped, a checksum so nothing is corrupted, and a range check so a tip target is never outside the allowed volume or further than 1 ms of motion from the last one. The **safety processor** is a second, simpler computer that reads the same encoders and can apply the brakes on its own.

Step 4. Label the arrows

Arrow	What travels	How often
Hand controls → Scale and filter	position and orientation of each hand, 6 numbers	1 kHz
Foot pedals → everything	clutch on/off, e-stop	immediate, wired
Console → Command validator	tip target pose, sequence number, checksum	1 kHz over a dedicated wire
Command validator → Joint control	a validated tip target, or "hold"	1 kHz
Joint control → Motors	a current per joint	1 kHz
Encoders → Joint control and Safety processor	two independent angle readings per joint	1 kHz, both boxes read both
Endoscope → Display	two 1080p video frames	60 Hz, encoded, under 43 ms end to end
Safety processor → Brakes	"engage"	immediate, a physical line
Everything → Audit log	commands, joint states, faults, video	1 kHz for data, continuous for video

Step 5. The three hardest arrows

Latency: video to the surgeon's eyes. The commands are fast (3 ms). What the surgeon feels is the video: 43 ms in the budget above. Every extra encode step, every Wi-Fi hop, every frame buffer adds to it, and past about 100 ms surgeons overshoot and slow down. So the video path is wired, uncompressed where possible, and measured on every build with a flashing light in front of the camera and a photodiode on the display.

Failure: any single part. The rule is that no single failure may cause motion. Each joint has two encoders (one absolute, one incremental); if they disagree by more than a small angle the safety processor brakes. Two controllers run the same code in lockstep and compare outputs. Power is dual. The watchdog freezes the arm 20 ms after the last valid command, so a cut cable means the tip moves 0.67 mm at most, and then holds still. Holding still, not retracting, is the safe state: a retract could tear tissue. The surgeon or nurse then removes the instrument by hand.

Correctness: the audit log. Every command and every joint reading at 1 kHz, 400 MB per operation, plus 29 GB of video, written to two disks, time-stamped from one clock, kept for years. If something goes wrong in theatre, this is the only way to know whether the robot did what it was told. It also feeds the regulator. The log writer must never slow the control loop, so it reads from a buffer the loop fills and the loop never waits for the disk.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
No autonomy at all	the surgeon is responsible and the regulator certifies a tool, not a decision maker	no "auto-suture", however tempting; the robot cannot be cleverer than its operator
Freeze, not retract, on fault	staying still cannot cause new injury; moving might	someone must remove the instrument by hand after a fault
Wired, dedicated links in the room	latency and reliability are certain; nothing shares the cable	no remote surgery over a network, which is a different, harder product
Dual encoders, dual controllers, separate safety processor	a single failure must never move the arm	roughly twice the electronics, and a lot of testing
3 to 1 scaling with a 6 Hz filter	precision matters more than speed for fine work	large moves need the clutch several times; surgeons switch scaling for coarse moves

Step 7. What I would build first

The command path with the validator and watchdog, on a bench, driving one joint, with a switch that cuts the cable. Prove the freeze is under 20 ms every time, thousands of times. Then the second encoder and the safety processor. Then the video path with a latency measurement. The audit log is in from day one because it is how you test everything else.

Questions someone might push on, with answers

- *Why not add a "no-go zone" so the tip cannot touch an organ?* That is autonomy: the robot overriding the surgeon. It also needs the robot to know where the organ is. Products do add simple bounds set by the surgeon before the case; that is a range check, not a decision.
- *Would you use ROS 2?* Not in the control or safety path. Certified medical software needs every line accounted for, and the 1 kHz loop needs guaranteed timing. ROS 2 would be fine for non-critical tooling: setup screens, the log viewer, simulation.
- *How is this different from the pick-and-place arm in 5.2?* Same joint loop, same idea of a separate safety box. The difference is who decides: 5.2 has a perception and planning stack that decides; this has a human, and the whole design exists to carry the human's intent faithfully and to prove afterwards that it did.

5.7 A self-driving shuttle on a fixed route

The request: "Design a small passenger shuttle that drives itself around a fixed 5 km loop with six stops, the kind Oxa and Aurigo have trialled in UK towns and airports."

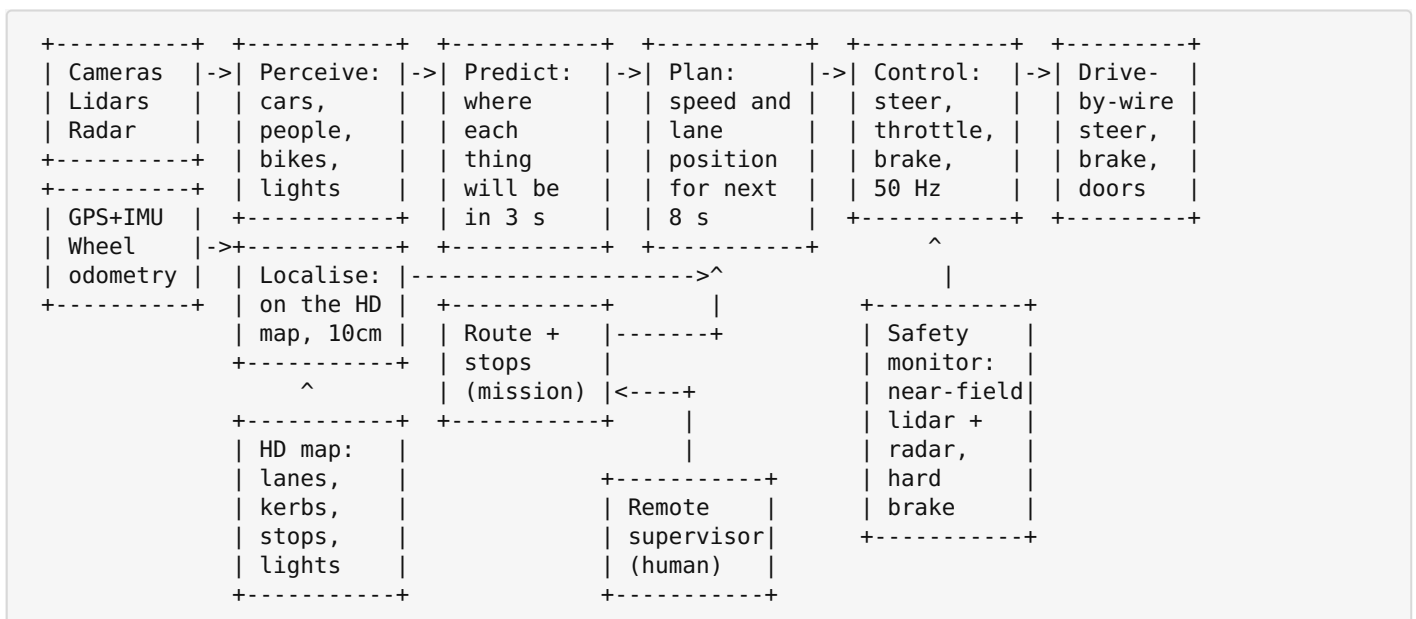
Step 1. What it must do and never do

Who and where: public roads at low speed, mixed with cars, cyclists and pedestrians; up to eight passengers. Done: the loop every 15 minutes, all day, stopping at each stop, at 25 km/h. Never: hit anything, run a red light, leave the route, move with a door open, strand passengers with no way to get help. Fixed: the route, which can be mapped in detail in advance; a vehicle with drive-by-wire (the computer can steer, brake and accelerate through electrical commands); a mobile network. Today: a bus with a driver. The question of whether a safety driver sits on board is answered in Step 6.

Step 2. Numbers first

Number	Value	Why it matters
Speed	25 km/h, 6.9 m/s	stopping distance and how far it moves during any delay
Loop	5 km, 12 minutes driving plus 6 stops of 30 s: 15 minutes, 4 trips an hour	the timetable
Stopping	hard brake 3 m/s squared: 8 m; plus 0.3 s of latency, 2.1 m: 10 m. Comfortable 2 m/s squared: 14 m	the shuttle must reliably see and classify things at 30 m, which it reaches in 4.3 s
Pedestrian prediction	3 s ahead: a walker at 1.5 m/s moves 4.5 m, a runner at 4 m/s moves 12 m	the size of the "might be there" zone in front of a person
Sensors	4 cameras 1080p at 20 Hz: 500 MB/s raw; 2 lidars of 32 beams: 1.3 million points a second each, 42 MB/s together	all on board; nothing raw leaves the vehicle
Localisation	20 Hz, error under 10 cm against the HD map	needed to stay in lane and stop at the kerb
Remote link	100 ms typical, sometimes seconds; the shuttle moves 0.7 m in 100 ms, 3.5 m in 500 ms	a remote person can advise, never steer

Step 3. Draw sense, think, act



The **HD map** (high definition map) is a detailed map of the route, made in advance: every lane edge, kerb, stop line, traffic light position, and the shape of the buildings and signs the lidar will see. **Localise** matches the live lidar against that shape to find the vehicle to 10 cm; GPS alone is metres out. **Predict** turns "a person at (x, y) moving at 1.5 m/s" into "a person somewhere in this 4.5 m long region over the next 3 s", and the planner keeps out of that region. **Radar** measures distance and speed directly and works in rain when cameras and lidar suffer.

Step 4. Label the arrows

Arrow	What travels	How often
Cameras → Perceive	four 1080p images	20 Hz

Arrow	What travels	How often
Lidars → Perceive, Localise, Safety monitor	point clouds	10 to 20 Hz
HD map → Localise and Plan	the map section around the vehicle, about 10 MB for the loop	loaded once, queried continuously
Localise → Plan and Predict	pose to 10 cm, with a "how sure"	20 Hz
Perceive → Predict	a list of objects: type, position, speed, size	20 Hz
Predict → Plan	for each object, the region it may occupy over 3 s	20 Hz
Mission → Plan	the next stop and the lane to follow	on change
Plan → Control	a trajectory: positions and speeds for the next 8 s	20 Hz
Control → Drive-by-wire	steering angle, throttle, brake	50 Hz
Shuttle → Remote supervisor	compressed video from one camera 2 to 4 Mbit/s, pose, state, and "I need a decision"	video on demand, state 1 Hz
Remote supervisor → Mission	"proceed", "wait", "use the other lane", "stop at next safe place"	on change
Safety monitor → Drive-by-wire	"brake now"	immediate, independent of everything above

Step 5. The three hardest arrows

Latency: something in front to the brakes. From 6.9 m/s the vehicle needs 10 m to stop hard, 14 m comfortably. The perceive, predict, plan chain takes 0.3 s and 2 m of travel; that is already in the 10 m. The safety monitor bypasses the chain: lidar and radar returns inside a box in front of the vehicle, scaled by speed, trigger the brake directly, with no classification. It brakes for a plastic bag too. That is accepted, because braking for a bag is embarrassing and hitting a child is not.

Failure: the remote supervisor and the link. A stuck shuttle (a parked van across the lane, an unclear roadworks layout) asks the supervisor. The supervisor sees the video and answers a question from a short list: proceed, wait, take the other lane, stop safely. The supervisor never steers, because at 100 ms to seconds of delay the vehicle has moved metres. If the link is down the shuttle waits in lane with hazard lights and passengers are told; it never guesses. If it is somewhere unsafe to wait it crawls to the next mapped safe stop.

Scale: the perception data and the map. 500 MB/s of camera data never leaves the vehicle; it is processed on board and only objects, at a few kB per frame, go to prediction. The HD map for a 5 km loop is small, about 10 MB, but it goes stale: roadworks move kerbs and cones. So the vehicle compares what it sees with the map every trip, and a mismatch (a kerb where there was none) both slows the vehicle and flags the map for an update by the mapping team.

Step 6. Trade-offs

Choice	Chosen because	At the cost of
Fixed route with an HD map	localisation to 10 cm and a known set of hazards; "anywhere" is a far harder product	every route change needs a mapping drive and a review
25 km/h, not 50	10 m stopping makes 30 m of reliable sensing enough; the safety case is provable	slower than a bus; only fits short urban loops
Safety monitor brakes for anything	nothing must depend on classification being right	false stops for bags, birds and heavy rain; passengers feel them

Choice	Chosen because	At the cost of
Remote supervisor advises, never drives	link delay makes driving unsafe; advice at decision points is safe	the shuttle waits when it needs a human and none is free
A safety driver on board during trials	the safety case is built from evidence, and the driver provides both a fallback and the evidence; UK trials have run this way while the law (the Automated Vehicles Act 2024) sets the route to removing the driver	the cost of a driver undoes the business case, so the design must record everything needed to justify removing them: every takeover, why, and what the system would have done

Step 7. What I would build first

The vehicle with the safety monitor and drive-by-wire only, driven by a person on the route, recording all sensors. That gives the map and a log to test perception against offline. Then localisation on the map, checked against the human drive. Then the full chain, first in replay, then on a closed road, then on the route with a safety driver. Takeovers are counted from the first day, because the number of miles between takeovers is the number that decides when the driver can leave.

Questions someone might push on, with answers

- *What about traffic lights?* The map says where each light is, so the camera looks in a small known region of the image, which is far more reliable than finding lights anywhere. Where possible the light also sends its state over a short-range radio, and the two must agree, or the vehicle stops.
- *What happens in fog?* Radar keeps working, cameras and lidar lose range. The planner reduces speed so that stopping distance fits inside what the sensors can still see, and below a threshold the shuttle goes to the next stop and waits. That is the same rule as the drone's wind limit in 5.4.
- *Why not a bigger, faster vehicle?* Stopping distance grows with the square of speed. At 50 km/h the hard stop is 32 m and the sensing requirement roughly doubles; the safety case becomes a different project.
- *Where does A or Dijkstra appear?** In the mission box, over the route graph (stops and junctions as nodes), the Dijkstra in tutorial/04-graphs-dijkstra.md. The lane-level planner is a different tool: it searches over short trajectories, not grid cells.

5.8 Common mistakes and how to avoid them

- **Starting with technology instead of requirements.** "I would use ROS 2 and a Jetson" before knowing the speed or the space. Fix: five questions, five numbers, then boxes. The technology falls out of the numbers.
- **Unlabelled arrows.** A line between two boxes with nothing on it is a question you have not answered. Fix: what travels, how often, on every arrow, even if you have to guess.
- **No numbers.** "Fast", "real-time", "lots of data". Fix: a number with a unit, rough is fine. 1.5 m/s, 20 Hz, 500 MB/s. Then check one of them against another: does the stopping distance fit inside the sensor range?
- **No failure story.** A design where every box works. Fix: for the three hardest arrows, say what happens when the arrow stops, and what the robot does about it.
- **Putting safety in the cloud, or in the clever code.** If the stop depends on the network or on perception being right, it is not a stop. Fix: a simple safety box on the robot with its own sensor and a wired line to the motors (5.2, 5.3, 5.6, 5.7 all have one).
- **Remote driving over a mobile link.** 300 ms is metres of travel. Fix: the robot drives, the remote human answers questions at decision points (5.3, 5.7).

- **One giant box called "AI".** Fix: split think into perceive, localise, predict, plan, control, each with a one-sentence job and an interface to the next.
 - **Forgetting the boring loops.** Battery, charging, logging, the map going stale, clock sync. Fix: 5.5's charging arithmetic (260 robots for 200 working) and 5.6's audit log are design decisions, not afterthoughts.
 - **Trade-offs stated as facts.** "Central planning is better." Fix: "Central because global optimality matters more than the single point of failure here, which we cover with a standby". Every choice has a cost; say it.
 - **No first step.** A design that needs everything before anything works. Fix: name the thinnest thing that tests the riskiest arrow, and build that (every Step 7 above).
-

5.9 A question bank

Fifteen more prompts, easiest first. For each, run the seven steps on paper in 40 minutes, then compare your hardest arrows and trade-offs with the six worked designs above.

1. A robot lawnmower for a garden with a pond.
 2. A window-cleaning robot for a three-storey office.
 3. A robot that restocks shelves in a supermarket at night.
 4. A robot that carries linen between floors in a hospital, using the lifts.
 5. A drone that counts sheep on a hillside farm.
 6. A robot arm that loads and unloads a CNC machine in a small workshop.
 7. A weeding robot that drives along crop rows and pulls weeds with a small arm.
 8. A fleet of ten cleaning robots in an airport terminal, sharing one map and one charging bay.
 9. An underwater robot that inspects the legs of an offshore platform, with no GPS and a slow acoustic link.
 10. A robot that sorts parcels by reading their labels on a conveyor, at two parcels a second.
 11. A quadruped robot that carries tools across a construction site with no flat floors.
 12. A humanoid that unloads boxes from a lorry, working next to people.
 13. A fleet of 50 agricultural robots on farms across a region, with one remote support team.
 14. A robotic taxi in a city centre with no fixed route and no safety driver.
 15. A robot that assists a surgeon by holding the endoscope and following the instruments, with the minimum autonomy needed to be useful and no more.
-

5.10 In your own words

A design question is a vague request, and the skill is turning it into requirements, numbers, boxes, labelled arrows, three hard arrows gone deep, trade-offs said as "X because Y matters more than Z", and a first thing to build. The same seven steps produced six very different robots above because the numbers were different: 1 m/s and a person's hand, 1.5 m/s and a 300 ms link, a 0.5 m gust against a 4 m gap, 200 robots and one clock, a surgeon's 43 ms of video, a shuttle's 10 m to stop. Safety always ended up in a small dumb box on the robot. The remote human always ended up answering questions, never steering.

Check yourself

1. What are the five questions to ask before drawing anything, and why does "what happens today without the robot" belong on the list?
2. Which five numbers do you write down first, and give one example from this level of a number that decided a design.

3. In 5.2, why is the light curtain 0.9 m outside the arm's reach and not 0.35 m?
 4. In 5.3, why can the remote supervisor not drive the robot, and what is the one number that says so?
 5. In 5.4, why is GPS not used to hold the 4 m standoff, and what is used instead?
 6. In 5.5, what single rule on each robot makes it safe to trust a central planner, and what does that rule cost?
 7. In 5.6, why is the safe state "freeze" rather than "retract", and how far can the tip move before the watchdog acts?
 8. In 5.7, what does the safety monitor brake for, and why is that accepted?
 9. State the central versus distributed planner trade-off in one "X because Y matters more than Z" sentence, for the warehouse in 5.5.
 10. Pick any prompt from 5.9. Without looking back, name its three hardest arrows and what you would build first.
-

5.11 Try it

Take prompt 8 from the bank (ten cleaning robots in an airport) and write the full seven steps in the same format as 5.2 to 5.7: five answers, five numbers, an ASCII diagram, an interface table, three hard arrows, a trade-off table, what to build first, and three questions someone might push on. Check every number you write with a few lines of Python, the way the numbers in this level were checked. When you have done it, do it again for prompt 14, and notice which arrows are the same and which are new.